

Como o sistema decide quais músicas mostrar

Guia falado do pipeline de recomendação: da base de faixas do Spotify até a resposta em linguagem natural. Tempos são sugestão — ajuste ao slot da banca.

~ 8-9 min no total

● motor determinístico (similaridade)

● LLM (linguagem)

0 Abertura 1 Base & features 2 Similaridade 3 Ranking 4 Perfil 5 LLM
6 Fechamento Mapa Perguntas

0 Abertura 0:30

- Contextualiza o projeto em 1 frase: assistente conversacional que recomenda música a partir de um pedido em linguagem natural.
- Coloca a pergunta que o roteiro inteiro responde: **quem escolhe a música — a IA generativa ou outra coisa?**

"Quando você pede pro assistente uma música mais animada, quem decide qual faixa aparece? Spoiler: não é o modelo de linguagem."

1 Base de dados e features de áudio 1:30 DETERMINÍSTICO

- **128.830 faixas** / 97.534 únicas / 118 gêneros, com **9 features contínuas** do Spotify: danceability, energy, loudness, speechiness, acousticness, instrumentalness, liveness, valence, tempo.
- Features categóricas (key, mode, time_signature) ficaram de fora da similaridade — viram filtro, não dimensão. Decisão veio de análise de correlação exploratória.
- Cada coluna é padronizada por **z-score**: $(x - \text{média}) / \text{desvio}$ — assim nenhuma feature domina só por ter escala maior (ex. tempo em BPM vs. valence de 0 a 1).

"Antes de comparar músicas, a gente precisa colocar todo mundo na mesma régua — por isso o z-score."

mostrar: recomendacao/dataset.py — trecho de padronização

2

Motor de similaridade (cosseno) 1:30 DETERMINÍSTICO

- Matriz de features padronizadas é normalizada uma única vez (L2); comparar uma música com outra vira **um produto escalar** — rápido mesmo com ~100 mil faixas.
- Buckets de humor/energia usados no pedido do usuário (baixa/média/alta, triste/neutro/feliz) viram um ponto de referência simples: z-score **-1, 0 ou +1** na dimensão certa.
- É uma convenção assumida no código, não um limiar validado estatisticamente — bom ponto pra citar como limitação conhecida.

"Cosseno mede o ângulo entre dois vetores de características — quanto mais próximo de 1, mais parecido o 'perfil sonoro' das duas músicas."

mostrar: recomendacao/indice.py — IndiceSimilaridade

3

Vetor-alvo, ranking e fallback 1:30 DETERMINÍSTICO

- O "alvo" da busca vem de três fontes possíveis: **centróide das faixas de um artista**, **vetor dos buckets** de humor/energia, ou o **perfil de gosto do usuário**.
- Quando existe perfil de usuário, o sistema mistura consulta e perfil **70% / 30%** — a busca pontual pesa mais que o histórico.
- Ranking = ordenar por similaridade decrescente, depois de aplicar filtros duros (gênero exato, sem duplicata, sem explicit se pedido).
- Sem nenhum sinal de busca, cai pra **popularity** bruto do Spotify. Se faltam faixas locais, completa via Spotify Search API — essas últimas não são ranqueadas por cosseno.

"O ranking nunca é mágica: é ordenar uma lista por um número calculado."

mostrar: `recomendacao/busca.py` — `_montar_vetor_alvo`, `_ranquear_por_similaridade`

4

Perfil de gosto do usuário 1:00 DETERMINÍSTICO

- Perfil = **centróide** (média) das features padronizadas das faixas do histórico Spotify da pessoa (top tracks, tocadas recentemente, salvas).
- Casamento com a base local é feito por `track_id` exato; se não encontra, cai pra nome + artista normalizado.
- É esse vetor que entra na mistura 70/30 da etapa anterior — o sistema "sabe" o gosto da pessoa sem precisar perguntar.

mostrar: `recomendacao/perfil.py`

5

Onde entra o LLM 1:30 LINGUAGEM

- Roteador por **regex** resolve pedidos simples ("mais animado", "pra dançar", gêneros comuns) sem gastar chamada de LLM.
- Quando o pedido é livre, **uma chamada de LLM** (qwen2.5:7b-instruct via Ollama, com fallback hospedado) só extrai um **JSON estruturado** — gênero, energia, valência, artista de referência etc. O LLM não vê nem ranqueia faixa nenhuma.

- Depois que o motor determinístico já escolheu as músicas, uma **segunda chamada** de LLM escreve o texto de resposta — proibida de citar qualquer faixa fora da lista já selecionada.
- Uma auditoria mecânica confere as citações no texto gerado contra a lista real, como rede de segurança contra alucinação.

"O modelo de linguagem entende o pedido e escreve a resposta — mas quem escolhe a música é sempre o cálculo de similaridade."

mostrar: chat/roteador.py, chat/extrator.py, chat/gerador.py, chat/auditoria.py

6 Fechamento 1:00

- Reforça o porquê da escolha: **sem modelo treinado** (nada de sklearn/tensorflow) — cosseno sobre features padronizadas é rápido, determinístico e **explicável**: dá pra apontar exatamente por que uma música foi escolhida.
- Cita a fonte de projeto: docs/PIPELINE_AGENTE_PROPOSTA_B.md (design) e analise_exploratoria.ipynb (escolha das 9 features).
- Abre pra perguntas.

"É simples de propósito: matemática decide o quê, linguagem decide o como dizer."

Mapa do código (backup pra perguntas)

Referência rápida se a banca pedir pra apontar onde algo acontece.

ARQUIVO	PAPEL
<code>recomendacao/dataset.py</code>	Carrega o dataset, define as 9 features e o z-score
<code>recomendacao/indice.py</code>	Matriz normalizada + similaridade de cosseno como produto escalar
<code>recomendacao/busca.py</code>	Monta o vetor-alvo, aplica filtros, ranqueia, faz fallback
<code>recomendacao/perfil.py</code>	Centróide do histórico Spotify do usuário

ARQUIVO	PAPEL
<code>recomendacao/historico_match.py</code>	Casa faixas do histórico com a base local
<code>recomendacao/spotify_fallback.py</code>	Completa resultados via Spotify Search API
<code>chat/roteador.py</code>	Regex: resolve pedidos simples sem chamar LLM
<code>chat/extrator.py</code>	LLM extrai JSON estruturado do pedido livre
<code>chat/gerador.py</code>	LLM escreve o texto da resposta final
<code>chat/auditoria.py</code>	Confere citações do texto contra as faixas reais
<code>chat/pipeline.py</code>	Orquestra: roteador → extrator → busca → gerador → auditoria

Perguntas prováveis da banca

Clique pra abrir a resposta preparada.

- Por que não usaram um modelo de machine learning treinado?

+
- Como garantem que o LLM não inventa uma música que não existe?

+
- O que acontece se a base local não tiver nada parecido?

+
- Por que a mistura é 70% consulta / 30% perfil do usuário?

+
- Por que essas 9 features e não outras (key, mode, time_signature)?

+
- Qual é o papel exato do LLM local (Ollama / qwen2.5)?

+

Escolha das features: `analise_exploratoria.ipynb` (repositório raiz).